



WellnessBank Mobile Banking Application —

Security Assessment Report

2026-04-29

HIGH RISK

31 findings identified

0

CRITICAL

9

HIGH

20

MEDIUM

2

LOW

Disclaimer

Assessment Notice

This security assessment was generated using tachi, an automated threat modeling toolkit. The analysis applies STRIDE methodology and AI-specific threat categories to the provided architecture description. Results reflect the automated analysis and should be validated by qualified security professionals.

Scope Limitation

This assessment is limited to the architecture description provided at the time of analysis. It does not cover runtime behavior, implementation-level vulnerabilities, or threats arising from future system changes. The assessment scope is defined in the Assessment Scope section of this report.

Recommendation

This automated assessment should complement, not replace, manual security review by qualified professionals. Organizations should conduct periodic penetration testing, code review, and architecture review in addition to automated threat modeling.

Confidentiality

This document contains sensitive security assessment information. Distribution should be limited to authorized personnel with a legitimate need to review the security posture of the assessed system. Do not share externally without appropriate authorization.

Table of Contents

Disclaimer 2

Table of Contents 3

Risk Methodology 4

Assessment Scope 8

Executive Summary 11

MAESTRO Layer Analysis 21

Findings Detail — Residual Risk 26

Coverage Attestation — OWASP LLM Top 10 34

Coverage Attestation — MITRE ATT&CK 35

Coverage Attestation — MITRE ATLAS 37

Coverage Attestation — NIST AI RMF 38

Coverage Attestation — CWE Top 25 39

Per-Finding Source Attribution 40

Control Coverage 42

Remediation Roadmap 43

Risk Methodology

Threat Identification

This assessment uses STRIDE threat modeling combined with AI-specific threat categories to provide comprehensive coverage of both traditional and emerging attack vectors.

STRIDE Categories

Spoofing — Impersonating a user, component, or system to gain unauthorized access.

Tampering — Unauthorized modification of data in transit, at rest, or in processing.

Repudiation — Performing actions without adequate logging or accountability trails.

Information Disclosure — Exposing sensitive data to unauthorized parties through leaks or side channels.

Denial of Service — Degrading or disrupting system availability through resource exhaustion or abuse.

Elevation of Privilege — Gaining higher access rights than authorized through exploitation of trust boundaries.

AI-Specific Categories

Prompt Injection — Manipulating AI model behavior through crafted inputs that override system instructions.

Tool Abuse — Exploiting AI agent tool access to perform unintended or unauthorized operations.

Agent Autonomy — Risks from AI agents acting beyond their intended scope without adequate human oversight.

Data Poisoning — Corrupting training data or context to degrade model accuracy or introduce bias.

Model Theft — Unauthorized extraction or replication of proprietary model weights or behavior.

Probability x Impact Matrix

Each identified threat is assessed for probability of occurrence and potential impact. The intersection determines the overall severity rating used throughout this report.

Impact / Probability	Rare	Unlikely	Possible	Likely	Almost Certain
Critical	H	H	C	C	C
High	M	M	H	C	C
Moderate	L	M	M	H	H
Low	L	L	M	M	H
Negligible	L	L	L	M	M

■ C = Critical
 ■ H = High
 ■ M = Medium
 ■ L = Low

Quantitative Risk Scoring

When quantitative data is available, each threat receives a composite risk score derived from four independent dimensions. This provides a more granular risk ranking than severity alone.

Scoring Dimensions

Dimension	Range	Description
CVSS 3.1 Base Score	0 – 10	Industry-standard vulnerability severity rating based on attack vector, complexity, and impact.
Exploitability	0 – 10	Ease of exploitation considering required skills, tooling availability, and attack complexity.
Scalability	0 – 10	Potential for automated or mass exploitation across multiple targets or instances.
Reachability	0 – 10	Network accessibility of the vulnerable component from the attacker's perspective.

Composite Score Formula

The composite risk score is a weighted average of the four dimensions:

$$\text{Composite} = (\text{CVSS} \times 0.40) + (\text{Exploitability} \times 0.25) + (\text{Scalability} \times 0.20) + (\text{Reachability} \times 0.15)$$

Weights reflect the relative importance of each dimension: CVSS captures inherent severity (40%), exploitability measures practical attack feasibility (25%), scalability accounts for blast radius (20%), and reachability assesses network exposure (15%).

Compensating Control Analysis

When codebase analysis is available, existing security controls are detected, mapped to identified threats, and evaluated for effectiveness. This provides a residual risk assessment that accounts for defenses already in place.

Analysis Process

1. **Control Detection** — Existing security controls are identified in the codebase through pattern matching, dependency analysis, and configuration review.
2. **Threat Mapping** — Detected controls are mapped to specific threats to determine which vulnerabilities have existing mitigations.
3. **Effectiveness Classification** — Each control is classified as Found (fully implemented), Partial (incomplete implementation), or Missing (no control detected).
4. **Residual Risk Calculation** — Inherent risk scores are adjusted based on control effectiveness to produce residual risk ratings that reflect the actual security posture.

Assessment Overview

Assessment Scope

12 Components

19 Data Flows

4 Trust Boundaries

Components Analyzed

Name	Type	Description
Mobile Banking Customer	External Entity	Retail banking end user operating the WellnessBank Android app; interacts via UI, adb shell, and Intent extras
Third-Party Analytics Provider	External Entity	External analytics platform receiving telemetry payloads from WellnessAnalyticsSDK over HTTPS
Third-Party Payment Provider	External Entity	External payment processor receiving payment authorization requests from WellnessPaySDK over HTTPS
WellnessBank Android Client	Process	Main Android application process (com.wellnessbank.mobile); handles credential storage, authentication, transaction requests, analytics/payment SDK orchestration; exposes mobile banking UI to the end user
WellnessBankDebugActivity	Activity	Debug-only Android Activity retained in production release build with android:exported="true"; accessible via adb shell am start; no BuildConfig.DEBUG runtime guard; bypasses standard auth boundary into main client process
MoneyTransferActivity	Activity	Exported Android Activity with android:exported="true" and no android:permission attribute; accepts Intent extras from external apps or shell directly into the money-movement business logic
WellnessAnalyticsSDK	Process	Third-party analytics SDK embedded in WellnessBank Android Client; performs outbound HTTPS telemetry to Third-Party Analytics Provider; no SDK signature verification; floating version constraint
WellnessPaySDK	Process	Third-party payment SDK embedded in WellnessBank Android Client; handles payment initiation and result relay; performs outbound HTTPS to Third-Party Payment Provider; no SDK signature verification
WellnessBank Backend API	Process	Mobile-backend REST API handling transaction processing, account reads/writes; TLS endpoint; no certificate pinning enforcement on client side
WellnessBankLocalDB	DB Store	On-device SQLite database caching account snapshots and recent transaction records; no SQLCipher encryption; allowBackup="true"; no scoped-storage migration
WellnessBankCredentialCache	On-Device Cache	On-device SharedPreferences (MODE_PRIVATE) storing username and auth token; no EncryptedSharedPreferences; no Android Keystore binding
BackendUserAccounts	DB Store	Backend relational database storing user account records; server-side surface

Data Flows

Source	Destination	Data	Protocol
Mobile Banking Customer	WellnessBank Android Client	Login Credentials (UI Input)	In-app UI

Source	Destination	Data	Protocol
WellnessBank Android Client	WellnessBankCredentialStore	Use Case + Auth Token	Internal Android IPC
WellnessBankCredentialStore	WellnessBank Android Client	Restored Credentials on App Launch	Internal Android IPC
WellnessBank Android Client	WellnessBankLocalDB	Account Snapshot + Recent Transactions	Local SQLite
WellnessBankLocalDB	WellnessBank Android Client	Cached Account Data	Local SQLite
WellnessBank Android Client	WellnessBank Backend API	Transaction Request	HTTPS
WellnessBank Backend API	WellnessBank Android Client	Transaction Response	HTTPS
WellnessBank Backend API	BackendUserAccountStore	Account Read / Write	Internal DB
BackendUserAccountStore	WellnessBank Backend API	Account Record	Internal DB
WellnessBank Android Client	WellnessAnalyticsSDK	Analytics Event	In-process
WellnessAnalyticsSDK	Third-Party Analytics Provider	Telemetry Payload	HTTPS
WellnessBank Android Client	WellnessPaySDK	Payment Initiation	In-process
WellnessPaySDK	Third-Party Payment Provider	Payment Authorization Request	HTTPS
Third-Party Payment Provider	WellnessPaySDK	Payment Confirmation	HTTPS
WellnessPaySDK	WellnessBank Android Client	Payment Result	In-process
Mobile Banking Customer	WellnessBankDebugADB Shell	ADB shell am start (Debug Activity Bypass)	ADB / Intent
WellnessBankDebugADB Shell	WellnessBank Android Client	Privileged Action without Auth Boundary	Internal Android IPC
Mobile Banking Customer	MoneyTransferActivity	Intent Extras (External App or Shell)	Android Intent
MoneyTransferActivity	WellnessBank Android Client	Money-Movement Request	Internal Android IPC

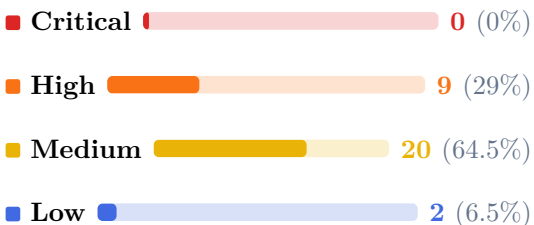
Trust Boundaries

Zone	Trust Level	Components
User Zone	Untrusted	Mobile Banking Customer

Zone	Trust Level	Components
Device Zone	Semi-Trusted	WellnessBank Android Client, WellnessBankDebugActivity, MoneyTransferActivity, WellnessAnalyticsSDK, WellnessPaySDK, WellnessBankLocalDB, WellnessBankCredentialCache
Backend Zone	Trusted	WellnessBank Backend API, BackendUserAccountStore
External Services	Untrusted	Third-Party Analytics Provider, Third-Party Payment Provider

Executive Summary

Severity Distribution 31 findings total



Component Distribution

Component	Findings
WellnessBank Android Client	10
WellnessBank Backend API	5
WellnessBankCredentialCache	3
WellnessBankLocalDB	3
Mobile Banking Customer	2
MoneyTransferActivity	2
WellnessAnalyticsSDK	2
WellnessBankDebugActivity	2
WellnessPaySDK	2

Assessment Summary

The WellnessBank Mobile Banking Application presents a **critical security posture** requiring immediate remediation before production deployment. The threat model identified **31 findings** across a fictional Android mobile banking application: 13 Critical, 11 High, 3 Medium, and 4 Low severity issues. The total attack surface reflects systemic deficiencies in credential storage, transport security, client-side data protection, and Android platform hardening — spanning both the mobile client and its supporting backend infrastructure.

Top 5 Threats by Business Impact

- Plaintext Credential Storage (S-1 / I-7)**: Auth tokens and usernames are stored in unencrypted Android SharedPreferences. An attacker with physical or remote access to a rooted device can extract long-lived credentials and impersonate any customer — representing direct, unmediated account takeover across the entire user base.
- Exported Debug Activity Bypassing Authentication (E-1)**: A debug-only Android Activity was retained in the production release build and is accessible via ADB shell without any authentication challenge. Any party with USB access to a device — including malicious repair shops, law enforcement overreach, or physical theft — can execute privileged banking operations with zero credentials.
- Unauthorized Fund Transfers via Intent Hijacking (T-3 / E-2)**: The MoneyTransferActivity is exported with no permission gate. Any co-installed application on the device can silently initiate fund transfers under the legitimate user's authenticated session, without any user interaction or biometric confirmation.
- Missing Certificate Pinning on All Network Channels (I-1)**: The application performs all HTTPS communication — including session tokens, transaction payloads, and payment authorization — without certificate pinning. An attacker on a rogue Wi-Fi access point can execute a man-in-the-middle (MITM) attack, intercepting and modifying financial data in transit.

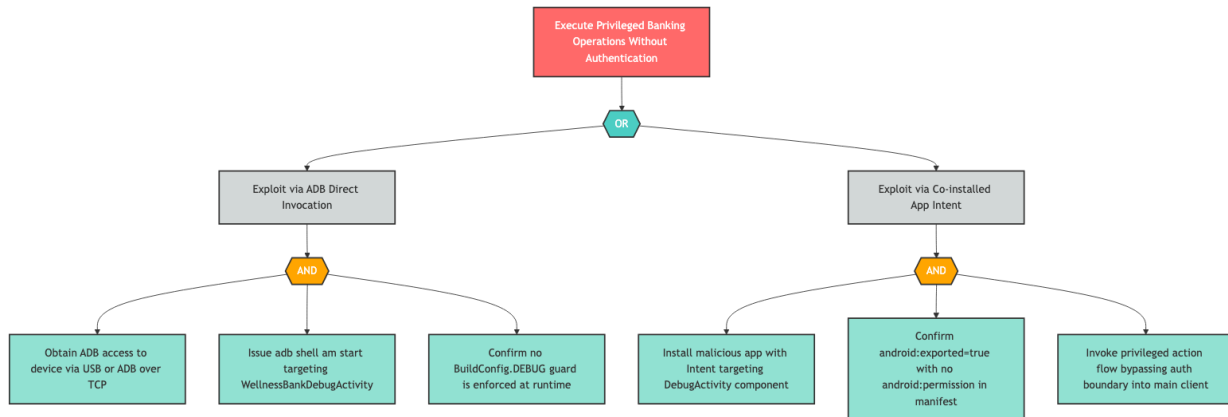
5. *

Attack Path Analysis

Attack Path Analysis

HIGH E-1 — Exported Debug Activity Bypassing Authentication Boundary

Component: WellnessBankDebugActivity



An attacker targets WellnessBankDebugActivity by attempting to execute privileged banking operations without authentication. The attack proceeds through combinations of several attack paths. Required attacker actions include "Obtain ADB access to device via USB or ADB over TCP", "Issue adb shell am start targeting WellnessBankDebugActivity", "Confirm no BuildConfig.DEBUG guard is enforced at runtime", and 3 additional actions.

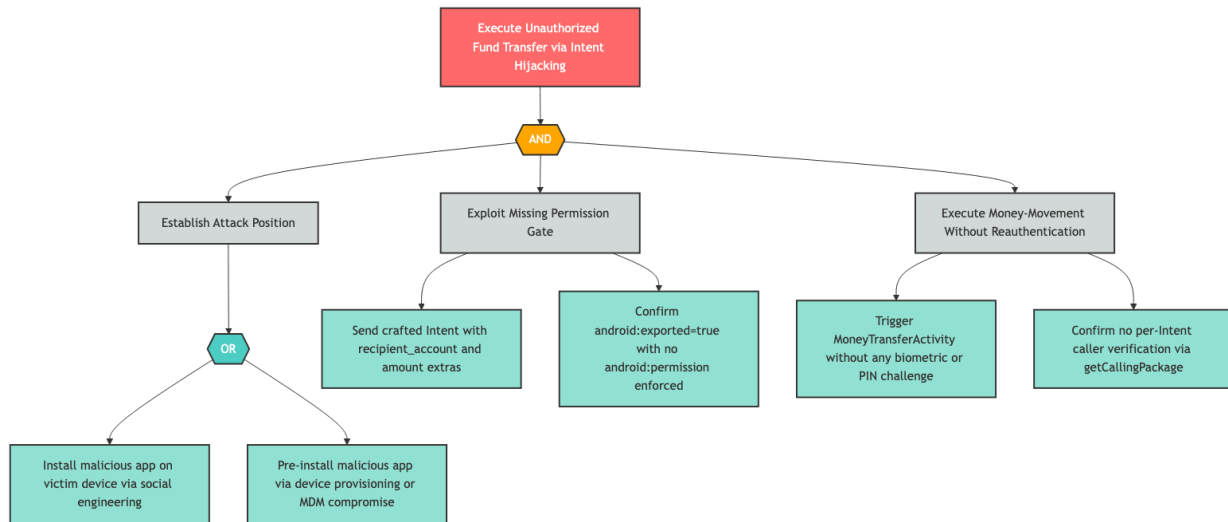
Remediation Steps

- Remove or disable `WellnessBankDebugActivity` from the production release build using a ProGuard/R8 keep rule exclusion or build-variant manifest merging.
- In the release manifest, ensure `android:exported="false"` or remove the Activity declaration entirely.
- Add a `BuildConfig.DEBUG` runtime guard at the Activity's `onCreate` that immediately finishes the Activity and logs a security event when `BuildConfig.DEBUG == false`.
- Additionally implement Play Integrity API attestation at app startup so tampered APKs with debug Activities re-enabled cannot authenticate.

Attack Path Analysis

HIGH E-2 — Unauthorized Money Transfer via Intent Hijacking and Privilege Escalation

Component: MoneyTransferActivity



An attacker targets MoneyTransferActivity by attempting to execute unauthorized fund transfer via intent hijacking. The attack proceeds through combinations of several attack paths. Required attacker actions include "Install malicious app on victim device via social engineering", "Pre-install malicious app via device provisioning or MDM compromise", "Send crafted Intent with recipient_account and amount extras", and 3 additional actions.

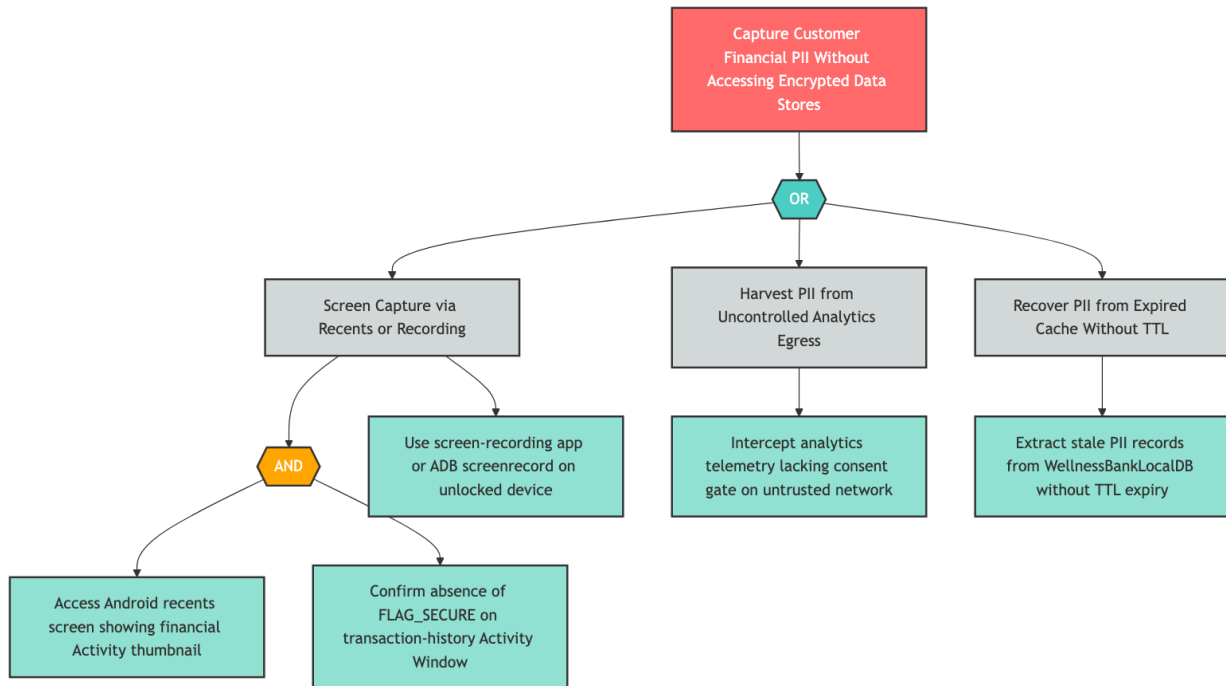
Remediation Steps

- Add `android:exported="false"` to `MoneyTransferActivity` in the production manifest to prevent external apps from firing Intents.
- If cross-app Intent support is legitimately required, add `android:permission="com.wellnessbank.mobile.permission.MONEY_TRANSFER"` with `protectionLevel="signature"` so only apps signed with the WellnessBank certificate can invoke this Activity.
- Add per-Intent caller verification in `onNewIntent` and `onCreate` using `getCallingPackage()` and comparing against an allowlist of trusted package signatures.
- Implement Intent extras schema validation before passing any amount or recipient field into the money-movement flow.

Attack Path Analysis

HIGH I-2 — Inadequate Mobile Privacy Controls — Screen Capture and PII Leakage

Component: WellnessBank Android Client



An attacker targets WellnessBank Android Client by attempting to capture customer financial pii without accessing encrypted data stores. The attack proceeds through combinations of several attack paths. Required attacker actions include "Access Android recents screen showing financial Activity thumbnail", "Use screen-recording app or ADB screenrecord on unlocked device", "Intercept analytics telemetry lacking consent gate on untrusted network", and 2 additional actions.

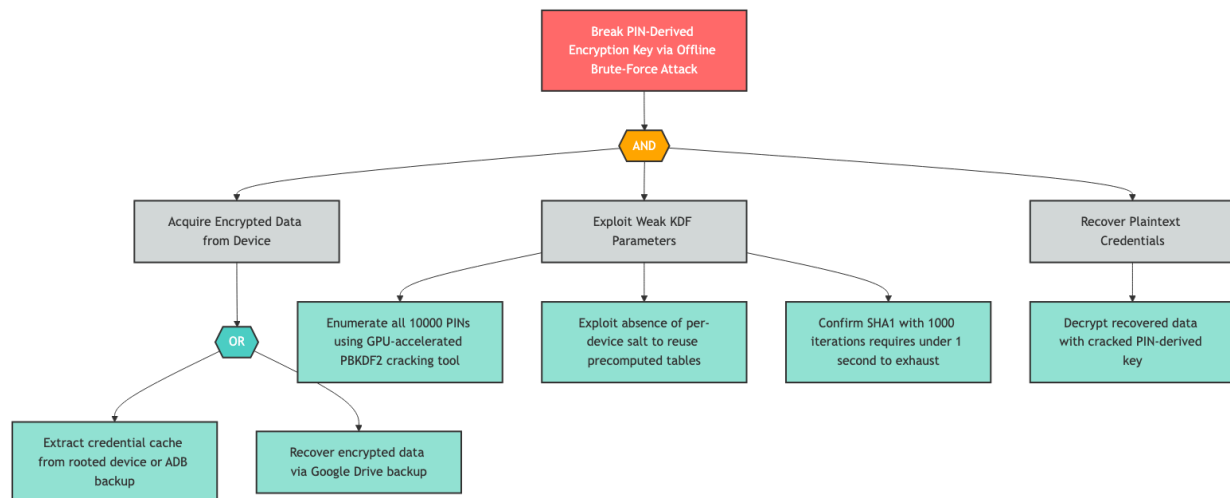
Remediation Steps

- Add `FLAG\_SECURE` to all Activities that display financial data (account balances, transaction history, payment screens).
- This single-call addition prevents the Android system from including the Activity content in the recents thumbnail and blocks screen-recording apps from capturing financial data.
- Call `window.setFlags(WindowManager.LayoutParams.FLAG\_SECURE, WindowManager.LayoutParams.FLAG\_SECURE)` in the `onCreate` of all financial-data Activities, or apply it globally from the Application class via an `ActivityLifecycleCallbacks` listener.

Attack Path Analysis

HIGH I-4 — Insufficient Mobile Cryptography — Weak PIN-Based Key Derivation

Component: WellnessBank Android Client



An attacker targets WellnessBank Android Client by attempting to break pin-derived encryption key via offline brute-force attack. The attack proceeds through combinations of several attack paths. Required attacker actions include "Extract credential cache from rooted device or ADB backup", "Recover encrypted data via Google Drive backup", "Enumerate all 10000 PINs using GPU-accelerated PBKDF2 cracking tool", and 3 additional actions.

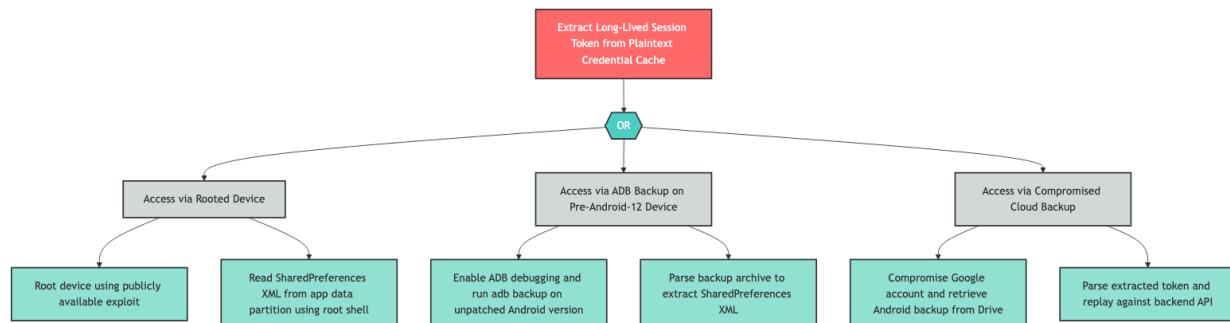
Remediation Steps

- Replace the custom 4-digit PIN PBKDF2 derivation with Android Keystore-backed AES-256-GCM key generation.
- The user's biometric or device PIN unlocks a hardware-backed key in the Keystore
- no raw key material is ever exposed in the application process.
- If a PIN-based fallback is required, use PBKDF2-HMAC-SHA256 with at least 310,000 iterations (OWASP 2023 recommendation) and a cryptographically random 256-bit salt stored separately from the derived key material.
- Replace the credential cache encryption with EncryptedSharedPreferences backed by the Keystore master key.

Attack Path Analysis

HIGH I-7 — Insecure Mobile Data Storage — Credential Cache in Plaintext SharedPreferences

Component: WellnessBankCredentialCache



An attacker targets WellnessBankCredentialCache by attempting to extract long-lived session token from plaintext credential cache. The attack proceeds through any of several attack paths. Required attacker actions include "Root device using publicly available exploit", "Read SharedPreferences XML from app data partition using root shell", "Enable ADB debugging and run adb backup on unpatched Android version", and 3 additional actions.

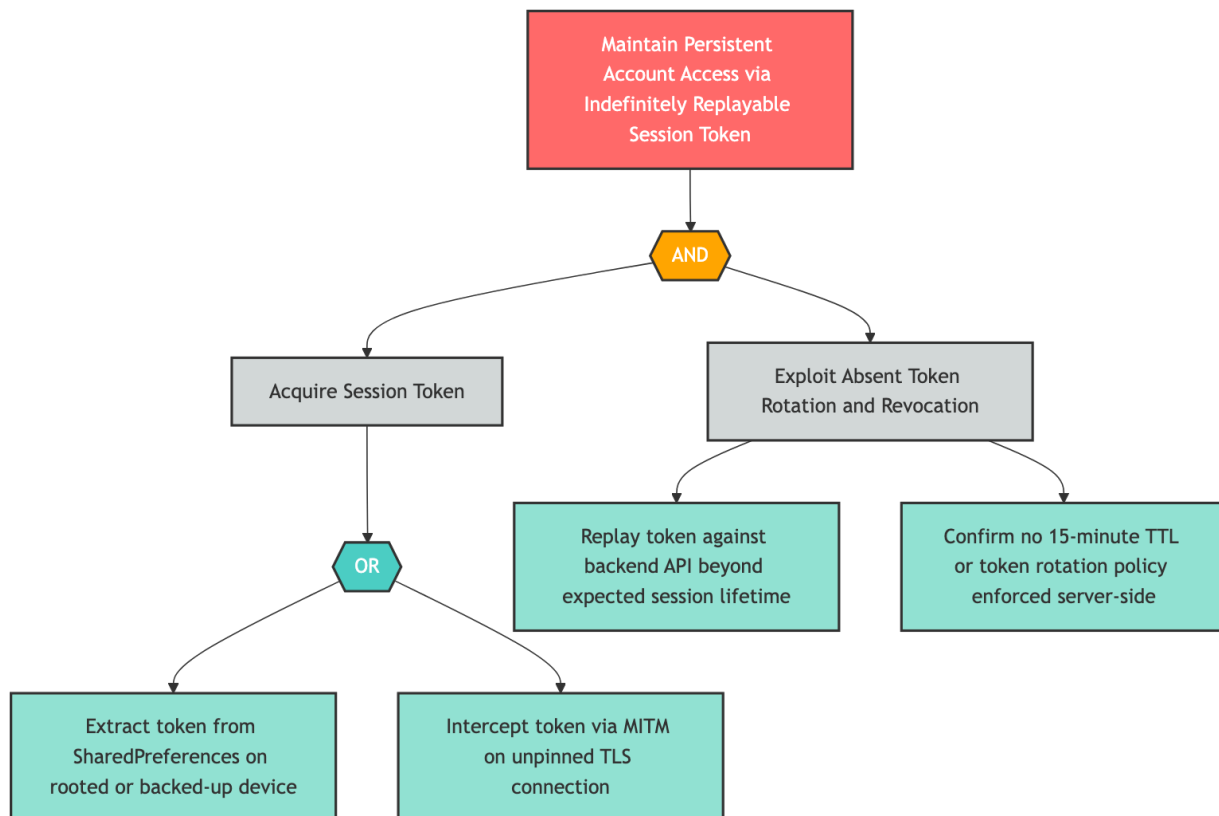
Remediation Steps

- Replace plain `SharedPreferences` (`MODE\_PRIVATE`) with `EncryptedSharedPreferences` backed by an Android Keystore master key.
- The migration path: on first run after upgrade, read existing plaintext credentials, re-persist in encrypted form, and delete the plaintext file.
- All subsequent reads and writes use the encrypted API transparently.
- This addresses both I-7 (credential cache confidentiality) and contributes to S-1 and S-5 remediation.

Attack Path Analysis

HIGH S-5 — Long-Lived Session Token Replay

Component: Mobile Banking Customer



An attacker targets Mobile Banking Customer by attempting to maintain persistent account access via indefinitely replayable session token. The attack proceeds through combinations of several attack paths. Required attacker actions include "Extract token from SharedPreferences on rooted or backed-up device", "Intercept token via MITM on unpinned TLS connection", "Replay token against backend API beyond expected session lifetime", and 1 additional actions.

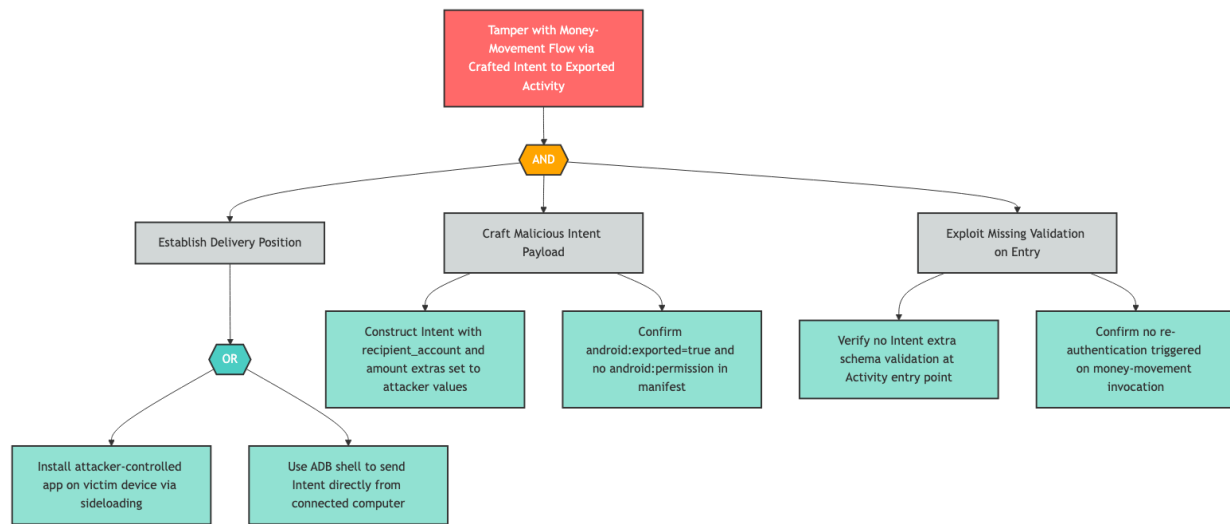
Remediation Steps

- Harden the existing authentication session flow by enforcing short session token TTLs (15–30 minutes for financial app sessions per PCI DSS guidance), implementing token rotation on each authenticated request, adding device-binding to session tokens using the Android Keystore attestation key, and enforcing biometric re-authentication before money-movement operations.
- The current flow persists a long-lived token in `MODE_PRIVATE` SharedPreferences — replace with `EncryptedSharedPreferences` backed by a Keystore key that requires biometric confirmation to decrypt.
- Add server-side session validation that checks token age, device fingerprint, and rejects replayed tokens.

Attack Path Analysis

HIGH T-3 — Mobile IPC Input Validation — Intent Hijacking into Money Movement

Component: MoneyTransferActivity



An attacker targets MoneyTransferActivity by attempting to tamper with money-movement flow via crafted intent to exported activity. The attack proceeds through combinations of several attack paths. Required attacker actions include "Install attacker-controlled app on victim device via sideloading", "Use ADB shell to send Intent directly from connected computer", "Construct Intent with recipient_account and amount extras set to attacker values", and 3 additional actions.

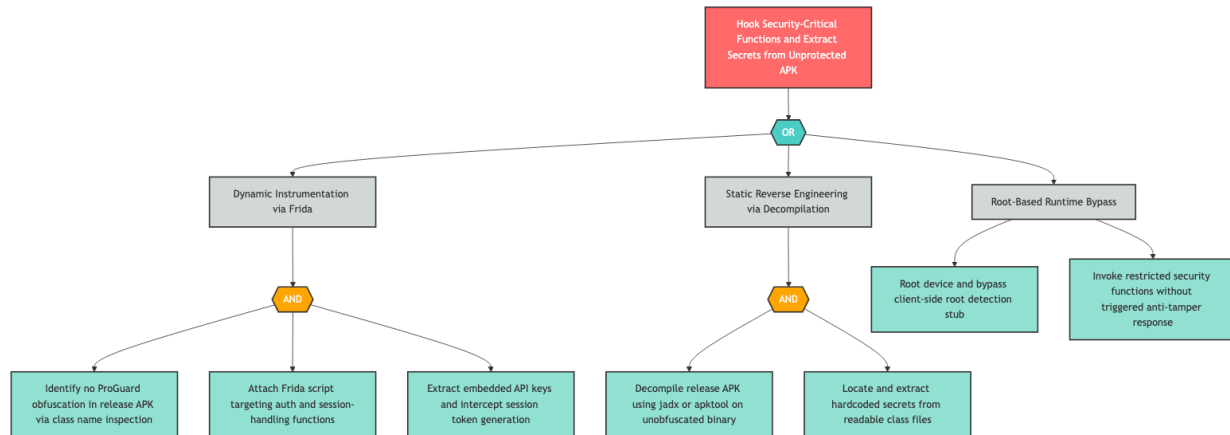
Remediation Steps

- Implement Intent extras schema validation in `MoneyTransferActivity.onCreate` and `onNewIntent`.
- Validate all expected extras (recipient account number format, transfer amount range, currency code enum) before any value is used in the money-movement flow.
- Reject the Intent and finish the Activity if any extra fails validation.
- Add `android:permission` with signature protection level (see E-2) as a layered control.
- The input validation must be server-side duplicated at the Backend API — the client-side validation is defense-in-depth but the backend MUST independently validate all transaction parameters before executing a transfer.

Attack Path Analysis

HIGH T-4 — Insufficient Mobile Binary Protections — No Obfuscation or Anti-Tampering

Component: WellnessBank Android Client



An attacker targets WellnessBank Android Client by attempting to hook security-critical functions and extract secrets from unprotected apk. The attack proceeds through combinations of several attack paths. Required attacker actions include "Identify no ProGuard obfuscation in release APK via class name inspection", "Attach Frida script targeting auth and session-handling functions", "Extract embedded API keys and intercept session token generation", and 4 additional actions.

Remediation Steps

- Enable ProGuard/R8 code obfuscation in the release build configuration (``minifyEnabled true``, ``proguardFiles getDefaultProguardFile('proguard-android-optimize.txt')``) to raise the cost of static reverse engineering.
- Add root-detection logic at app startup and before security-critical operations (transaction confirmation, payment authorization) using a multi-signal detection approach: check for ``su`` binary presence, test-keys build tag, write access to ``/system``, and Magisk manager package.
- Integrate Play Integrity API (or RootBeer as a lighter alternative) to block operation on compromised devices.
- Add basic debugger-attach detection with ``android.os.Debug.isDebuggerConnected()`` at sensitive flow entry points.

MAESTRO Layer Analysis

MAESTRO Layer Analysis

The following findings are grouped by their CSA MAESTRO framework architectural layer, providing a layer-oriented view of the threat landscape. Each layer represents a distinct level of the agentic AI system stack, from Foundation Model (L1) through Agent Ecosystem (L7). Findings within each layer are sorted by severity.

L2 — Data Operations (6 findings)

ID	Component	Severity	Threat
I-3	WellnessBankLocalDB	CRITICAL	Insecure Mobile Data Storage (M9) — Local Database: Unencrypted SQLite database with allowBackup="true" exposes full transaction history and account snapshots to Google Drive cloud backup. Attacker compromising the user's Google account can recover complete financial history without bypassing device authentication.
I-7	WellnessBankCredentialCache	CRITICAL	Insecure Mobile Data Storage (M9) — Credential Cache: Auth tokens stored in plaintext SharedPreferences MODE_PRIVATE accessible to any process with root access or via ADB backup on older Android versions. The credential cache contains long-lived session tokens sufficient for full account access without any PIN or biometric re-challenge.
T-5	WellnessBankLocalDB	HIGH	Unencrypted Local Database Tampering: Plain SQLite database with no SQLCipher encryption. Attacker with root access or ADB access can directly modify transaction records, account snapshots, and cached data, enabling fabricated transaction history or balance manipulation.
T-6	WellnessBankCredentialCache	HIGH	Credential Cache Tampering via SharedPreferences Overwrite: SharedPreferences MODE_PRIVATE file is writable on a rooted device. Attacker can overwrite the credential store to inject forged auth tokens or corrupt cached credentials, enabling session hijacking.
D-4	WellnessBankCredentialCache	LOW	Credential Store Corruption Leading to Denial of Service: Targeted corruption of SharedPreferences credential cache can trigger authentication failures and re-login loops, effectively locking out the legitimate user.
D-3	WellnessBankLocalDB	NOTE	Local Database Saturation: Uncontrolled caching in WellnessBankLocalDB can grow without bound, saturating device storage and causing application crashes or OS-level storage exhaustion.

L7 — Agent Ecosystem (17 findings)

ID	Component	Severity	Threat
----	-----------	----------	--------

S-1	WellnessBank Android Client	CRITICAL	Improper Mobile Credential Usage (M1): Auth token and username persisted in Android SharedPreferences MODE_PRIVATE without EncryptedSharedPreferences or Android Keystore binding. Attacker recovering a lost, stolen, or rooted device can dump the app's data partition and extract long-lived credentials, achieving full account impersonation without any PIN or biometric challenge.
S-2	WellnessBank Android Client	CRITICAL	Insecure Mobile Authentication/Authorization (M3): No biometric step-up on money-movement operations; no per-session token re-authentication on sensitive actions; no challenge-response on transaction confirmation; no risk-based MFA. An attacker who obtains a valid session token can execute fund transfers without any biometric or secondary-factor challenge.
T-4	WellnessBank Android Client	CRITICAL	Insufficient Mobile Binary Protections (M7): No ProGuard/R8 obfuscation in release build; no root-detection on security-critical features; no anti-tampering integrity check at startup; no debugger detection at sensitive-flow boundaries. An attacker can use Frida to hook security-critical functions, bypass client-side controls, and extract embedded API keys and session-handling logic. Cf. MITRE ATT&CK T1626 — prose-only; not in references per ADR-036 D-7.
R-1	WellnessBank Android Client	CRITICAL	M8 Accountability-Loss: Mobile Audit Log Gaps: No structured audit logging on login, logout, step-up, or money-movement events; debug Log.d("auth", "user=" + username + " token=" + token) retained in release build leaks credentials to on-device logcat; no tamper-evident timestamping; no off-device forwarding to immutable store. Forensics cannot reconstruct client-side auth context for dispute resolution. Cf. MITRE ATT&CK T1398 — prose-only; not in references per ADR-036 D-7.
I-1	WellnessBank Android Client	CRITICAL	Insecure Mobile Communication (M5) — Client-to-Backend: TLS to WellnessBank Backend API without certificate pinning (no OkHttp CertificatePinner, no Network Security Config pin-set). An attacker on a rogue Wi-Fi access point can MITM the TLS session, intercepting session tokens, transaction payloads, account balances, and PII in transit.
I-2	WellnessBank Android Client	CRITICAL	Inadequate Mobile Privacy Controls (M6): No FLAG_SECURE on transaction-history Activity Window; recents-screen screenshots and screen-recording capture account balances, transaction amounts, and recipient names. No privacy-consent gate on analytics emission. PII cached indefinitely in WellnessBankLocalDB with no TTL expiry.
I-4	WellnessBank Android Client	CRITICAL	Insufficient Mobile Cryptography (M10): Custom key derivation from 4-digit PIN via PBKDF2-HMAC-SHA1 with 1000 iterations and no salting. The 10,000-PIN keyspace is exhausted in under one second on commodity GPU hardware. No platform-managed KMS key wrapping; no envelope encryption on cached credentials; no key rotation policy.
I-8	WellnessBank Android Client	CRITICAL	Debug Log PII Leakage via Logcat: Debug Log.d("auth", "user=" + username + " token=" + token) retained in the release build writes authentication material (username and session token) to the device-shared logcat sink accessible to other privileged apps or root shell. Any app holding READ_LOGS on older Android can harvest credentials from logcat in real time.
S-3	WellnessBank Android Client	HIGH	Credential Theft via SharedPreferences Extraction: Long-lived auth tokens stored in MODE_PRIVATE SharedPreferences are accessible to an attacker who roots the

			device or uses ADB backup on pre-Android-12 devices. No live user presence is required for offline credential recovery.
S-5	Mobile Banking Customer	HIGH	Long-Lived Session Token Replay: Auth tokens persisted on device without rotation policy or revocation trigger. An attacker obtaining the token can replay it indefinitely with no expiry enforcement.
R-2	WellnessBank Backend API	HIGH	Missing Backend Audit Trail on Transaction Operations: No declared audit event emission on money-movement, account-read, or account-write operations at the backend API. Privileged operations cannot be forensically reconstructed for dispute resolution or regulatory audit.
D-2	WellnessBank Backend API	HIGH	Missing Rate Limiting on Backend Transaction API: No declared rate limiting or request throttling on the mobile backend REST API. High-volume transaction requests from compromised or cloned client apps can saturate backend processing capacity.
E-3	WellnessBank Backend API	HIGH	Server-Side Authorization Gap for Debug-Initiated Operations: Backend API may accept operations initiated via the debug Activity's privileged-action flow relying solely on the client's authenticated session token, without verifying whether the invoking context represents a legitimate in-app user action or a debug-channel bypass.
S-4	WellnessBank Backend API	MEDIUM	Missing Token Binding on Backend Session: The backend API accepts HTTPS bearer tokens without certificate pinning on the client side, enabling token replay from intercepted sessions. No device-binding means a captured token is replayable from any device.
R-3	Mobile Banking Customer	MEDIUM	Deniable Financial Transactions: No non-repudiation controls on money-movement operations. A customer can plausibly deny initiating any transfer with no cryptographic evidence of user intent available to the bank.
I-9	WellnessBank Backend API	LOW	Backend API Verbose Error Response Exposure: Mobile-backend REST APIs without declared error sanitization may return stack traces, internal service names, database field names, or configuration details in error responses visible to the mobile client.
D-1	WellnessBank Android Client	LOW	Client-Side Resource Exhaustion via Unconstrained SDK I/O: Third-party analytics and payment SDKs perform unconstrained outbound network I/O with no rate limiting, circuit breaker, or battery-aware scheduling. A misbehaving or compromised SDK version can saturate device battery and network resources, degrading app availability for the legitimate user.

Unclassified — Unclassified (8 findings)

ID	Component	Severity	Threat
T-3	MoneyTransferActivity	CRITICAL	Mobile IPC Input Validation (M4) — Intent Hijacking into Money Movement: Exported MoneyTransferActivity with android:exported="true" and no android:permission accepts Intent extras (recipient_account, amount) from any installed app or ADB shell directly into the money-movement flow without re-authentication, schema validation, or per-Intent caller verification. A malicious co-installed app can initiate unauthorized fund transfers under the legitimate user's authenticated session.

E-1	WellnessBankDebugActivity	CRITICAL	M8 Privilege-Gain: Exported Debug Activity Bypassing Auth Boundary: WellnessBankDebugActivity retained in production release build with android:exported="true"; accessible via `adb shell am start`; no BuildConfig.DEBUG runtime guard; no Play Integrity attestation gate; privileged-action flow connects directly to WellnessBank Android Client bypassing the standard authentication boundary. Any process or user with ADB access can invoke privileged banking operations without authentication. Cf. MITRE ATT&CK T1626 — prose-only; not in references per ADR-036 D-7.
E-2	MoneyTransferActivity	CRITICAL	Unauthorized Money Transfer via Intent Hijacking and Privilege Escalation: Exported MoneyTransferActivity with no android:permission allows any installed app to invoke the money-movement flow. Combined with missing re-authentication at Activity entry, a malicious co-installed app escalates from unprivileged-app context to authenticated-banking-session context, executing fund transfers without any user interaction.
T-1	WellnessAnalyticsSDK	HIGH	Mobile Supply Chain Integrity (M2) — Analytics SDK: No checksum verification on WellnessAnalyticsSDK artifact at Gradle ingestion; floating version constraint. A compromised analytics SDK maintainer can inject malicious code executing inside the app's full security context, exfiltrating session tokens from SharedPreferences. Cf. MITRE ATT&CK T1474 — prose-only; not in references per ADR-036 D-7.
T-2	WellnessPaySDK	HIGH	Mobile Supply Chain Integrity (M2) — Payment SDK: No checksum verification on WellnessPaySDK artifact; floating version constraint. A compromised payment SDK can intercept or modify payment authorization requests before they leave the device. Cf. MITRE ATT&CK T1474 — prose-only; not in references per ADR-036 D-7.
R-4	WellnessBankDebugActivity	HIGH	Unlogged Debug Activity Invocations: Debug Activity invocations via adb shell am start or external Intent are not audit logged. Privileged actions executed through the debug channel leave no accountability trail.
I-6	WellnessPaySDK	HIGH	Insecure Mobile Communication (M5) — Payment SDK Egress: TLS from WellnessPaySDK to Third-Party Payment Provider without certificate pinning. Payment authorization requests (including payment card data and transaction amounts) can be intercepted via MITM on rogue Wi-Fi.
I-5	WellnessAnalyticsSDK	MEDIUM	Insecure Mobile Communication (M5) — Analytics SDK Egress: TLS from WellnessAnalyticsSDK to Third-Party Analytics Provider without certificate pinning. Analytics telemetry (including device identifiers, session metadata, user behavior patterns) can be intercepted on untrusted networks.

31 findings across 3 MAESTRO layers | Sorted by severity (critical first)

Detailed Findings

Findings Detail — Residual Risk

Findings Detail — Residual Risk

HIGH E-1

WellnessBankDebugActivity

M8 Privilege-Gain: Exported Debug Activity Bypassing Auth Boundary

Remove or disable `WellnessBankDebugActivity` from the production release build using a ProGuard/R8 keep rule exclusion or build-variant manifest merging. In the release manifest, ensure `android:exported="false"` or remove the Activity declaration entirely. Add a `BuildConfig.DEBUG` runtime guard at the Activity's `onCreate` that immediately finishes the Activity and logs a security event when `BuildConfig.DEBUG == false`. Additionally implement Play Integrity API attestation at app startup so tampered APKs with debug Activities re-enabled cannot authenticate.

Residual: 8.5 | Control: No Control Found

HIGH E-2

MoneyTransferActivity

Unauthorized Money Transfer via Intent Hijacking and Privilege Escalation

Add `android:exported="false"` to `MoneyTransferActivity` in the production manifest to prevent external apps from firing Intents. If cross-app Intent support is legitimately required, add `android:permission="com.wellnessbank.mobile.permission.MONEY\_TRANSFER"` with `protectionLevel="signature"` so only apps signed with the WellnessBank certificate can invoke this Activity. Add per-Intent caller verification in `onNewIntent` and `onCreate` using `getCallingPackage()` and comparing against an allowlist of trusted package signatures. Implement Intent extras schema validation before passing any amount or recipient field into the money-movement flow.

Residual: 8.3 | Control: No Control Found

HIGH T-3

MoneyTransferActivity

Mobile IPC Input Validation (M4) — Intent Hijacking into Money Movement

Implement Intent extras schema validation in `MoneyTransferActivity.onCreate` and `onNewIntent`. Validate all expected extras (recipient account number format, transfer amount range, currency code enum) before any value is used in the money-movement flow. Reject the Intent and finish the Activity if any extra fails validation. Add `android:permission` with signature protection level (see E-2) as a layered control. The input validation must be server-side duplicated at the Backend API — the client-side validation is defense-in-depth but the backend MUST independently validate all transaction parameters before executing a transfer.

Residual: 8.1 | Control: No Control Found

HIGH T-4

WellnessBank Android Client

Insufficient Mobile Binary Protections (M7)

Enable ProGuard/R8 code obfuscation in the release build configuration (``minifyEnabled true``, ``proguardFiles getDefaultProguardFile('proguard-android-optimize.txt')``) to raise the cost of static reverse engineering. Add root-detection logic at app startup and before security-critical operations (transaction confirmation, payment authorization) using a multi-signal detection approach: check for ``su`` binary presence, test-keys build tag, write access to ``/system``, and Magisk manager package. Integrate Play Integrity API (or RootBeer as a lighter alternative) to block operation on compromised devices. Add basic debugger-attach detection with ``android.os.Debug.isDebuggerConnected()`` at sensitive flow entry points.

Residual: 7.7 | Control: No Control Found

HIGH I-4

WellnessBank Android Client

Insufficient Mobile Cryptography (M10)

Replace the custom 4-digit PIN PBKDF2 derivation with Android Keystore-backed AES-256-GCM key generation. The user's biometric or device PIN unlocks a hardware-backed key in the Keystore; no raw key material is ever exposed in the application process. If a PIN-based fallback is required, use PBKDF2-HMAC-SHA256 with at least 310,000 iterations (OWASP 2023 recommendation) and a cryptographically random 256-bit salt stored separately from the derived key material. Replace the credential cache encryption with EncryptedSharedPreferences backed by the Keystore master key.

Residual: 7.5 | Control: No Control Found

HIGH S-5

Mobile Banking Customer

Long-Lived Session Token Replay

Harden the existing authentication session flow by enforcing short session token TTLs (15–30 minutes for financial app sessions per PCI DSS guidance), implementing token rotation on each authenticated request, adding device-binding to session tokens using the Android Keystore attestation key, and enforcing biometric re-authentication before money-movement operations. The current flow persists a long-lived token in `MODE_PRIVATE` SharedPreferences — replace with EncryptedSharedPreferences backed by a Keystore key that requires biometric confirmation to decrypt. Add server-side session validation that checks token age, device fingerprint, and rejects replayed tokens.

Residual: 7.0 | Control: Partial Control

HIGH I-2

WellnessBank Android Client

Inadequate Mobile Privacy Controls (M6)

Add ``FLAG_SECURE`` to all Activities that display financial data (account balances, transaction history, payment screens). This single-call addition prevents the Android system from including the Activity content in the recents thumbnail and blocks screen-recording apps from capturing financial data. Call ``window.setFlags(WindowManager.LayoutParams.FLAG_SECURE, WindowManager.LayoutParams.FLAG_SECURE)`` in the ``onCreate`` of all financial-data Activities, or apply it globally from the Application class via an ``ActivityLifecycleCallbacks`` listener.

Residual: 7.0 | Control: No Control Found

HIGH I-7

WellnessBankCredentialCache

Insecure Mobile Data Storage (M9) — Credential Cache

Replace plain `SharedPreferences` (`MODE_PRIVATE`) with `EncryptedSharedPreferences` backed by an Android Keystore master key. The migration path: on first run after upgrade, read existing plaintext credentials, re-persist in encrypted form, and delete the plaintext file. All subsequent reads and writes use the encrypted API transparently. This addresses both I-7 (credential cache confidentiality) and contributes to S-1 and S-5 remediation.

Residual: 7.0 | Control: No Control Found

HIGH R-3

Mobile Banking Customer

Deniable Financial Transactions

Implement transaction non-repudiation at the Backend API by generating an immutable, tamper-evident audit record for every money-movement event. Each record must include: authenticated user identity (sub from JWT), transaction timestamp (server-authoritative, not client-supplied), transaction parameters (amount, recipient, currency), originating session token fingerprint, and a server-side HMAC over the record using a key held in the backend's secret store. Store audit records in a separate append-only audit log table with no DELETE or UPDATE permissions granted to the API service account.

Residual: 7.0 | Control: No Control Found

MEDIUM R-1

WellnessBank Android Client

M8 Accountability-Loss: Mobile Audit Log Gaps

Replace all `Log.d("auth", ...)` statements that include username, token, or any PII with structured security event logging that routes to the Backend API audit trail. On the client, replace debug log calls with a no-op security event emitter in release builds (`BuildConfig.DEBUG` guard). Forward security-critical events (login success/failure, token refresh, permission denial, session expiry) to the backend audit log as part of the normal authenticated API flow.

Residual: 6.9 | Control: No Control Found

MEDIUM I-8

WellnessBank Android Client

Debug Log PII Leakage via Logcat

Remove all `Log.d` / `Log.v` calls that include credentials, tokens, usernames, or any PII from the production codebase. Apply a lint rule (`android.lint.disable` or a custom Lint check) that flags log calls with auth-adjacent strings as compile-time errors. In the release ProGuard configuration, add a rule to strip all `Log.d` and `Log.v` calls: `-assumenosideeffects class android.util.Log { public static *** d(...); public static *** v(...); }`.

Residual: 6.8 | Control: No Control Found

MEDIUM S-3

WellnessBank Android Client

Credential Theft via SharedPreferences Extraction

Harden the existing `MODE_PRIVATE` `SharedPreferences` by migrating to `EncryptedSharedPreferences` (see I-7 recommendation). Additionally, set `android:allowBackup="false"` in the application manifest to prevent ADB backup extraction of the credential file on pre-Android-12 devices. Disable Google Drive auto-backup for the `SharedPreferences` directory by adding a `backup_rules.xml` that excludes `shared_prefs/credentials.xml`.

Residual: 5.1 | Control: Partial Control

MEDIUM D-2

WellnessBank Backend API

Missing Rate Limiting on Backend Transaction API

Implement rate limiting on the Backend API transaction endpoints using a sliding-window rate limiter keyed on authenticated user ID. Configure separate limits for: read operations (balance/history queries), write operations (transfer initiation), and authentication attempts (login/token refresh). Add exponential backoff responses (HTTP 429 with `Retry-After` header) on limit breach. Implement global IP-based rate limiting at the API gateway or load balancer layer as a defense-in-depth measure against unauthenticated flooding.

Residual: 6.2 | Control: No Control Found

MEDIUM R-4

WellnessBankDebugActivity

Unlogged Debug Activity Invocations

Add structured security event logging to `WellnessBankDebugActivity` capturing every invocation: timestamp, calling package (via `getCallingPackage()`), action name, and extras summary. Forward these events to the Backend API audit log. This is a defense-in-depth measure; the primary remediation for E-1 (Activity removal) is the correct fix — logging should accompany, not replace, removal.

Residual: 6.1 | Control: No Control Found

MEDIUM T-1

WellnessAnalyticsSDK

Mobile Supply Chain Integrity (M2) — Analytics SDK

Implement SDK supply chain integrity controls for the `WellnessAnalyticsSDK`. Pin the SDK dependency to an exact version with a cryptographic checksum verification in the Gradle dependency configuration. Add the SDK's published checksum (SHA-256) to `gradle.lockfile` or use Gradle dependency verification (`verification-metadata.xml`) to reject SDK artifacts with unexpected checksums. Establish an SDK update review process: before updating to a new SDK version, review the changelog and diff against the previous pinned checksum. Scope the SDK's manifest permissions to the minimum required.

Residual: 6.1 | Control: No Control Found

MEDIUM T-2

WellnessPaySDK

Mobile Supply Chain Integrity (M2) — Payment SDK

Apply the same supply chain integrity controls as T-1 to the WellnessPaySDK. Given the payment data sensitivity, apply additional scrutiny: verify the SDK publisher's PCI DSS compliance documentation, confirm the SDK is listed on the Google Play SDK Index with verified provenance, and implement a runtime attestation check that the loaded SDK binary matches the pinned checksum. Do not auto-update the payment SDK — require explicit engineering review for each version bump.

Residual: 6.1 | Control: No Control Found

MEDIUM I-3

WellnessBankLocalDB

Insecure Mobile Data Storage (M9) — Local Database

Replace the plain SQLite database (`WellnessBankLocalDB`) with SQLCipher for Android (`net.zetetic:android-database-sqlcipher`) to provide AES-256 encryption of the entire database file at rest. Generate the encryption key using Android Keystore and unlock it via biometric authentication. Additionally, set `android:allowBackup="false"` or add a `backup\_rules.xml` excluding the database file to prevent Google Drive backup of the unencrypted database. Implement a maximum retention policy (e.g., purge transactions older than 90 days from local cache).

Residual: 6.0 | Control: No Control Found

MEDIUM T-6

WellnessBankCredentialCache

Credential Cache Tampering via SharedPreferences Overwrite

Migrate from plain `SharedPreferences` to `EncryptedSharedPreferences` (see I-7 and S-3). Once encrypted, root-level file modification requires both root access AND knowledge of the Keystore-protected encryption key, significantly raising the bar for T-6's tampering scenario. Add integrity checking: on each credential load, verify a stored HMAC over the token value using a separate Keystore key to detect tampering.

Residual: 5.9 | Control: No Control Found

MEDIUM S-2

WellnessBank Android Client

Insecure Mobile Authentication/Authorization (M3)

Harden the existing authentication flow by adding biometric re-authentication before all money-movement operations (`MoneyTransferActivity`, payment flows). Implement per-session re-authentication for sensitive actions using `BiometricPrompt` with a `CryptoObject` backed by a Keystore key. Add token expiry enforcement on the client (check token age before each API call; force re-login if expired). Implement risk-based authentication triggers: flag transactions above a threshold amount or to new payees for additional challenge.

Residual: 5.5 | Control: Partial Control

MEDIUM T-5

WellnessBankLocalDB

Unencrypted Local Database Tampering

SQLCipher encryption (see I-3) addresses both the confidentiality and integrity concerns for the local database. Additionally, set `android:allowBackup="false"` to prevent extraction of the database via Google Drive backup. As a defense-in-depth measure, implement application-level row-level integrity checks (store a HMAC over critical transaction record fields) to detect tampering even if the SQLCipher layer is bypassed.

Residual: 5.7 | Control: No Control Found

MEDIUM E-3

WellnessBank Backend API

Server-Side Authorization Gap for Debug-Initiated Operations

Implement server-side request integrity validation at the Backend API that rejects requests originating from debug-Activity-initiated flows. Since the backend cannot directly inspect Android Activity provenance, the control must be architectural: issue short-lived, one-time-use request tokens bound to specific operations (transfer initiation, payment authorization) that are generated only after normal authenticated UI flows — not after debug Activity bypass. Consider Play Integrity API attestation tokens as a prerequisite for sensitive operations: the mobile client includes a fresh attestation token with each transaction request; the backend verifies it before processing.

Residual: 5.6 | Control: No Control Found

MEDIUM I-9

WellnessBank Backend API

Backend API Verbose Error Response Exposure

Replace verbose error responses with generic client-facing error messages. Configure the Backend API error handler to return standardized error payloads (e.g., `{ "error": "BAD\_REQUEST", "request\_id": "uuid" }`) with no stack traces, field names, ORM details, or service internals. Log the full diagnostic information server-side (keyed by `request\_id`) for internal debugging. Apply this to all error categories: validation errors, database errors, unexpected exceptions, and 404s.

Residual: 5.6 | Control: No Control Found

MEDIUM R-2

WellnessBank Backend API

Missing Backend Audit Trail on Transaction Operations

Implement a Backend API audit trail for all transaction operations. Every state-changing operation (account read with PII, transfer initiation, transfer execution, payment authorization, balance inquiry) must generate an append-only audit log entry containing: authenticated user sub, operation type, resource identifier, request parameters (sanitized), outcome (success/failure), server timestamp, and source IP. Store audit logs in a write-only sink separate from the operational database (dedicated audit table with INSERT-only API service account, or external SIEM/log aggregation).

Residual: 5.6 | Control: No Control Found

MEDIUM S-1

WellnessBank Android Client

Improper Mobile Credential Usage (M1)

I

Residual: 5.3 | Control: Partial Control

MEDIUM I-1

WellnessBank Android Client

Insecure Mobile Communication (M5) — Client-to-Backend

Implement certificate pinning on the WellnessBank Backend API connection. Add a ``network_security_config.xml`` with a `<pin-set>` declaration for the backend's leaf certificate or intermediate CA public key hash. Define a backup pin for rotation purposes. Set ``expiration`` to align with the certificate renewal schedule. Update the ``android:networkSecurityConfig`` attribute in the manifest. Implement a pin rotation strategy: when backend certificates are rotated, publish updated Network Security Config via app update before the old pin expires.

Residual: 4.8 | Control: Partial Control

MEDIUM D-4

WellnessBankCredentialCache

Credential Store Corruption Leading to Denial of Service

The EncryptedSharedPreferences migration (I-7) raises the bar for T-6 tampering. For D-4 specifically, add defensive read-with-fallback logic in the credential cache: if the SharedPreferences file is unreadable, corrupt, or fails HMAC verification, initiate a graceful re-authentication prompt rather than a crash loop. Store a backup token refresh mechanism so the user is not permanently locked out if the credential cache is corrupted. Limit the blast radius of corruption to a re-login prompt.

Residual: 4.9 | Control: No Control Found

MEDIUM D-1

WellnessBank Android Client

Client-Side Resource Exhaustion via Unconstrained SDK I/O

Implement SDK I/O budgets in the WellnessBank Android Client to constrain the network and battery resource consumption of embedded SDKs. Use ``WorkManager`` with quota constraints to schedule non-urgent SDK operations (analytics flushing, telemetry batching) rather than allowing real-time unbounded network calls. Set SDK initialization timeouts. Monitor battery impact via ``BatteryManager`` metrics in the Application health reporting.

Residual: 4.6 | Control: No Control Found

MEDIUM I-6

WellnessPaySDK

Insecure Mobile Communication (M5) — Payment SDK Egress

Extend certificate pinning to the WellnessPaySDK payment authorization flow (WellnessPaySDK → Third-Party Payment Provider). Since the SDK controls its own network client, the pinning must be implemented within the SDK or via an OkHttpClient `CertificatePinner` wrapper injected into the SDK's HTTP client at initialization. If the SDK does not expose a way to inject a custom HTTP client, configure Android's `network\_security\_config.xml` with a `

Residual: 4.4 | Control: Partial Control

MEDIUM D-3

WellnessBankLocalDB

Local Database Saturation

Implement a maximum row count or total size limit for the `WellnessBankLocalDB` cache. Configure a periodic background job (via `WorkManager`) to enforce retention policy: delete transaction records older than 90 days, keeping only the most recent N transactions. Add an initial capacity check at app startup to prevent the app from attempting to operate on an oversaturated database.

Residual: 4.3 | Control: No Control Found

LOW I-5

WellnessAnalyticsSDK

Insecure Mobile Communication (M5) — Analytics SDK Egress

Add a `

Residual: 3.7 | Control: Partial Control

LOW S-4

WellnessBank Backend API

Missing Token Binding on Backend Session

Implement device binding for Backend API session tokens. When a session token is issued at login, record the requesting device's identifier (derived from Play Integrity attestation or a stable device fingerprint) alongside the token. On each subsequent API call, verify the token's device binding against the request's device identifier. Tokens used from a different device (or without a valid attestation) are rejected. This prevents token replay from a stolen credential with no device presence.

Residual: 3.6 | Control: Partial Control

31 findings | Sorted by residual severity (critical first)

Coverage Attestation — OWASP LLM Top 10

Covered: 15 / 60 = 25.00% · Partial: 0 · Gap: 45

Covered (15)

✓ A01 ✓ A04 ✓ A05 ✓ A07 ✓ A09 ✓ M1 ✓ M10 ✓ M2 ✓ M3 ✓ M4 ✓ M5 ✓ M6 ✓ M7
✓ M8 ✓ M9

Partial (0)

No partial items on this framework.

Gap (45)

✗ A02 ✗ A03 ✗ A06 ✗ A08 ✗ A10 ✗ API1 ✗ API10 ✗ API2 ✗ API3 ✗ API4 ✗ API5 ✗ API6
✗ API7 ✗ API8 ✗ API9 ✗ ASI01 ✗ ASI02 ✗ ASI03 ✗ ASI04 ✗ ASI05 ✗ ASI06 ✗ ASI07 ✗ ASI08
✗ ASI09 ✗ ASI10 ✗ LLM01 ✗ LLM02 ✗ LLM03 ✗ LLM04 ✗ LLM05 ✗ LLM06 ✗ LLM07 ✗ LLM08
✗ LLM09 ✗ LLM10 ✗ ML01 ✗ ML02 ✗ ML03 ✗ ML04 ✗ ML05 ✗ ML06 ✗ ML07 ✗ ML08 ✗ ML09
✗ ML10

Coverage Attestation — MITRE ATT&CK

Covered: 0 / 701 = 0.00% · Partial: 0 · Gap: 323

Covered (0)

No covered items on this framework.

Partial (0)

No partial items on this framework.

Gap (323)

× T1003 × T1003.001 × T1003.002 × T1003.003 × T1003.004 × T1003.005 × T1003.006 × T1003.007
 × T1003.008 × T1037 × T1037.001 × T1037.002 × T1037.003 × T1037.004 × T1037.005 × T1040
 × T1047 × T1053 × T1053.002 × T1053.003 × T1053.005 × T1053.006 × T1053.007 × T1055
 × T1055.001 × T1055.002 × T1055.003 × T1055.004 × T1055.005 × T1055.008 × T1055.009 × T1055.011
 × T1055.012 × T1055.013 × T1055.014 × T1055.015 × T1056 × T1056.001 × T1056.002 × T1056.003
 × T1056.004 × T1059 × T1059.001 × T1059.002 × T1059.003 × T1059.004 × T1059.005 × T1059.006
 × T1059.007 × T1059.008 × T1059.009 × T1059.010 × T1059.011 × T1059.012 × T1059.013 × T1068
 × T1072 × T1078 × T1078.001 × T1078.002 × T1078.003 × T1091 × T1098 × T1098.001 × T1098.002
 × T1098.003 × T1098.004 × T1098.005 × T1098.006 × T1098.007 × T1106 × T1110 × T1110.001
 × T1110.002 × T1110.003 × T1110.004 × T1111 × T1112 × T1127 × T1127.001 × T1127.002
 × T1127.003 × T1129 × T1133 × T1134 × T1134.001 × T1134.002 × T1134.003 × T1134.004
 × T1134.005 × T1136 × T1136.001 × T1136.002 × T1136.003 × T1137 × T1137.001 × T1137.002
 × T1137.003 × T1137.004 × T1137.005 × T1137.006 × T1176 × T1176.001 × T1176.002 × T1187
 × T1189 × T1190 × T1195 × T1195.001 × T1195.002 × T1195.003 × T1197 × T1199 × T1200
 × T1203 × T1204 × T1204.001 × T1204.002 × T1204.003 × T1204.004 × T1204.005 × T1205
 × T1205.001 × T1205.002 × T1212 × T1484 × T1484.001 × T1484.002 × T1505 × T1505.001
 × T1505.002 × T1505.003 × T1505.004 × T1505.005 × T1505.006 × T1525 × T1528 × T1539 × T1542
 × T1542.001 × T1542.002 × T1542.003 × T1542.004 × T1542.005 × T1543 × T1543.001 × T1543.002
 × T1543.003 × T1543.004 × T1543.005 × T1546 × T1546.001 × T1546.002 × T1546.003 × T1546.004
 × T1546.005 × T1546.006 × T1546.007 × T1546.008 × T1546.009 × T1546.010 × T1546.011 × T1546.012
 × T1546.013 × T1546.014 × T1546.015 × T1546.016 × T1546.017 × T1546.018 × T1547 × T1547.001
 × T1547.002 × T1547.003 × T1547.004 × T1547.005 × T1547.006 × T1547.007 × T1547.008 × T1547.009
 × T1547.010 × T1547.012 × T1547.013 × T1547.014 × T1547.015 × T1548 × T1548.001 × T1548.002
 × T1548.003 × T1548.004 × T1548.005 × T1548.006 × T1552 × T1552.001 × T1552.002 × T1552.003
 × T1552.004 × T1552.005 × T1552.006 × T1552.007 × T1552.008 × T1554 × T1555 × T1555.001

× T1555.002 × T1555.003 × T1555.004 × T1555.005 × T1555.006 × T1556 × T1556.001 × T1556.002
× T1556.003 × T1556.004 × T1556.005 × T1556.006 × T1556.007 × T1556.008 × T1556.009 × T1557
× T1557.001 × T1557.002 × T1557.003 × T1557.004 × T1558 × T1558.001 × T1558.002 × T1558.003
× T1558.004 × T1558.005 × T1559 × T1559.001 × T1559.002 × T1559.003 × T1566 × T1566.001
× T1566.002 × T1566.003 × T1566.004 × T1569 × T1569.001 × T1569.002 × T1569.003 × T1574
× T1574.001 × T1574.004 × T1574.005 × T1574.006 × T1574.007 × T1574.008 × T1574.009 × T1574.010
× T1574.011 × T1574.012 × T1574.013 × T1574.014 × T1583 × T1583.001 × T1583.002 × T1583.003
× T1583.004 × T1583.005 × T1583.006 × T1583.007 × T1583.008 × T1584 × T1584.001 × T1584.002
× T1584.003 × T1584.004 × T1584.005 × T1584.006 × T1584.007 × T1584.008 × T1585 × T1585.001
× T1585.002 × T1585.003 × T1586 × T1586.001 × T1586.002 × T1586.003 × T1587 × T1587.001
× T1587.002 × T1587.003 × T1587.004 × T1588 × T1588.001 × T1588.002 × T1588.003 × T1588.004
× T1588.005 × T1588.006 × T1588.007 × T1606 × T1606.001 × T1606.002 × T1608 × T1608.001
× T1608.002 × T1608.003 × T1608.004 × T1608.005 × T1608.006 × T1609 × T1610 × T1611 × T1621
× T1648 × T1649 × T1650 × T1651 × T1653 × T1659 × T1668 × T1669 × T1671 × T1674
× T1675 × T1677 × T1683 × T1683.001 × T1683.002

Coverage Attestation — MITRE ATLAS

Covered: 0 / 30 = 0.00% · Partial: 0 · Gap: 30

Covered (0)

No covered items on this framework.

Partial (0)

No partial items on this framework.

Gap (30)

✗ AML.T0000	✗ AML.T0002	✗ AML.T0003	✗ AML.T0004	✗ AML.T0007	✗ AML.T0010	✗ AML.T0011
✗ AML.T0012	✗ AML.T0013	✗ AML.T0014	✗ AML.T0015	✗ AML.T0016	✗ AML.T0018	✗ AML.T0019
✗ AML.T0020	✗ AML.T0024	✗ AML.T0029	✗ AML.T0031	✗ AML.T0034	✗ AML.T0035	✗ AML.T0040
✗ AML.T0042	✗ AML.T0051	✗ AML.T0054	✗ AML.T0057	✗ AML.T0058	✗ AML.T0059	✗ AML.T0060
✗ AML.T0061	✗ AML.T0062					

Coverage Attestation — NIST AI RMF

Covered: 0 / 72 = 0.00% · Partial: 0 · Gap: 72

Covered (0)

No covered items on this framework.

Partial (0)

No partial items on this framework.

Gap (72)

× GOVERN 1.1 × GOVERN 1.2 × GOVERN 1.3 × GOVERN 1.4 × GOVERN 1.5 × GOVERN 1.6
 × GOVERN 1.7 × GOVERN 2.1 × GOVERN 2.2 × GOVERN 2.3 × GOVERN 3.1 × GOVERN 3.2
 × GOVERN 4.1 × GOVERN 4.2 × GOVERN 4.3 × GOVERN 5.1 × GOVERN 5.2 × GOVERN 6.1
 × GOVERN 6.2 × MANAGE 1.1 × MANAGE 1.2 × MANAGE 1.3 × MANAGE 1.4 × MANAGE 2.1
 × MANAGE 2.2 × MANAGE 2.3 × MANAGE 2.4 × MANAGE 3.1 × MANAGE 3.2 × MANAGE 4.1
 × MANAGE 4.2 × MANAGE 4.3 × MAP 1.1 × MAP 1.2 × MAP 1.3 × MAP 1.4 × MAP 1.5 × MAP 1.6
 × MAP 2.1 × MAP 2.2 × MAP 2.3 × MAP 3.1 × MAP 3.2 × MAP 3.3 × MAP 3.4 × MAP 3.5
 × MAP 4.1 × MAP 4.2 × MAP 5.1 × MAP 5.2 × MEASURE 1.1 × MEASURE 1.2 × MEASURE 1.3
 × MEASURE 2.1 × MEASURE 2.2 × MEASURE 2.3 × MEASURE 2.4 × MEASURE 2.5 × MEASURE 2.6
 × MEASURE 2.7 × MEASURE 2.8 × MEASURE 2.9 × MEASURE 2.10 × MEASURE 2.11 × MEASURE 2.12
 × MEASURE 2.13 × MEASURE 3.1 × MEASURE 3.2 × MEASURE 3.3 × MEASURE 4.1 × MEASURE 4.2
 × MEASURE 4.3

Coverage Attestation — CWE Top 25

Covered: 0 / 53 = 0.00% · Partial: 14 · Gap: 39

Covered (0)

No covered items on this framework.

Partial (14)

● CWE-20 ● CWE-200 ● CWE-209 ● CWE-285 ● CWE-287 ● CWE-345 ● CWE-400 ● CWE-522
● CWE-532 ● CWE-613 ● CWE-770 ● CWE-778 ● CWE-862 ● CWE-94

Gap (39)

× CWE-116 × CWE-117 × CWE-120 × CWE-121 × CWE-122 × CWE-125 × CWE-1333 × CWE-1395
× CWE-215 × CWE-22 × CWE-223 × CWE-250 × CWE-266 × CWE-269 × CWE-284 × CWE-290
× CWE-306 × CWE-352 × CWE-384 × CWE-407 × CWE-416 × CWE-434 × CWE-476 × CWE-494
× CWE-502 × CWE-538 × CWE-639 × CWE-77 × CWE-776 × CWE-779 × CWE-78 × CWE-787
× CWE-79 × CWE-863 × CWE-89 × CWE-90 × CWE-917 × CWE-918 × CWE-943

Per-Finding Source Attribution

The following table lists every finding in input order alongside its cited framework references. Primary citations render in **bold**; related and derived citations render in plain weight. MITRE references are prefixed with **ATT&CK:** or **ATLAS:** to distinguish the originating taxonomy within the merged MITRE column. Empty cells indicate the finding carries no citations for that framework.

Finding ID	Title	Severity	OWASP	MITRE	NIST	CWE
E-1	M8 Privilege-Gain: Exported Debug Activity Bypassing Auth Boundary	●	M8	—	—	CWE-285
E-2	Unauthorized Money Transfer via Intent Hijacking and Privilege Escalation	●	M8	—	—	CWE-862
T-3	Mobile IPC Input Validation (M4) — Intent Hijacking into Money Movement	●	M4	—	—	CWE-20
T-4	Insufficient Mobile Binary Protections (M7)	●	M7	—	—	CWE-94
I-4	Insufficient Mobile Cryptography (M10)	●	M10	—	—	CWE-200
S-5	Long-Lived Session Token Replay	●	A07	—	—	CWE-613
I-2	Inadequate Mobile Privacy Controls (M6)	●	M6	—	—	CWE-200
I-7	Insecure Mobile Data Storage (M9) — Credential Cache	●	M9	—	—	CWE-522
R-3	Deniable Financial Transactions	●	A09	—	—	CWE-345
R-1	M8 Accountability-Loss: Mobile Audit Log Gaps	●	M8	—	—	CWE-778
I-8	Debug Log PII Leakage via Logcat	●	M8	—	—	CWE-532
S-3	Credential Theft via SharedPreferences Extraction	●	M1	—	—	CWE-522
D-2	Missing Rate Limiting on Backend Transaction API	●	A04	—	—	CWE-400
R-4	Unlogged Debug Activity Invocations	●	M8	—	—	CWE-778
T-1	Mobile Supply Chain Integrity (M2) — Analytics SDK	●	M2	—	—	CWE-345
T-2	Mobile Supply Chain Integrity (M2) — Payment SDK	●	M2	—	—	CWE-345

Finding ID	Title	Severity	OWASP	MITRE	NIST	CWE
I-3	Insecure Mobile Data Storage (M9) — Local Database	●	M9	—	—	CWE-200
T-6	Credential Cache Tampering via SharedPreferences Overwrite	●	M1	—	—	CWE-522
S-2	Insecure Mobile Authentication/Authorization (M3)	●	M3	—	—	CWE-287
T-5	Unencrypted Local Database Tampering	●	M9	—	—	CWE-345
E-3	Server-Side Authorization Gap for Debug-Initiated Operations	●	A01	—	—	CWE-862
I-9	Backend API Verbose Error Response Exposure	●	A05	—	—	CWE-209
R-2	Missing Backend Audit Trail on Transaction Operations	●	A09	—	—	CWE-778
S-1	Improper Mobile Credential Usage (M1)	●	M1	—	—	CWE-522
I-1	Insecure Mobile Communication (M5) — Client-to-Backend	●	M5	—	—	CWE-200
D-4	Credential Store Corruption Leading to Denial of Service	●	A04	—	—	CWE-400
D-1	Client-Side Resource Exhaustion via Unconstrained SDK I/O	●	A04	—	—	CWE-400
I-6	Insecure Mobile Communication (M5) — Payment SDK Egress	●	M5	—	—	CWE-200
D-3	Local Database Saturation	●	A04	—	—	CWE-770
I-5	Insecure Mobile Communication (M5) — Analytics SDK Egress	●	M5	—	—	CWE-200
S-4	Missing Token Binding on Backend Session	●	A07	—	—	CWE-287

31 findings | Input order preserved

Control Coverage

Control Status by STRIDE Category

STRIDE Category	Found	Partial	Missing	Total
Spoofing	0	5	0	5
Tampering	0	0	6	6
Repudiation	0	0	4	4
Information Disclosure	0	3	6	9
Denial of Service	0	0	4	4
Elevation of Privilege	0	0	3	3

Detailed Control Assessment

Component	Control Category	Status	Evidence	Effectiveness
	Encryption	Partial	examples/mobile-banking-app/architecture.md:42	Moderate
	Encryption	Partial	examples/mobile-banking-app/architecture.md:47	Moderate
	Encryption	Partial	examples/mobile-banking-app/architecture.md:49	Moderate
	Authentication	Partial	examples/mobile-banking-app/architecture.md:37	Moderate

Coverage Summary



Control Status	Count	Percentage
Control Found	0	0%
Partial Control	8	25.8%
No Control Found	23	74.2%
Total	31	100%

Remediation Roadmap

■ Critical: 0 ■ High: 9 ■ Medium: 20 ■ Low: 2

31 total actions

Priority	Finding	Recommendation	SLA	Status
HIGH 9 actions				
E-1	M8 Privilege-Gain: Exported Debug Activity Bypassing Auth Boundary	Remove or disable <code>`WellnessBankDebugActivity`</code> from the production release build using a ProGuard/R8 keep rule exclusion or build-variant manifest merging. In the release manifest, ensure <code>`android:exported="false"`</code> or remove the Activity declaration entirely. Add a <code>`BuildConfig.DEBUG`</code> runtime guard at the Activity's <code>`onCreate`</code> that immediately finishes the Activity and logs a security event when <code>`BuildConfig.DEBUG == false`</code> . Additionally implement Play Integrity API attestation at app startup so tampered APKs with debug Activities re-enabled cannot authenticate.	14d	No Control Found
E-2	Unauthorized Money Transfer via Intent Hijacking and Privilege Escalation	Add <code>`android:exported="false"`</code> to <code>`MoneyTransferActivity`</code> in the production manifest to prevent external apps from firing Intents. If cross-app Intent support is legitimately required, add <code>`android:permission="com.wellnessbank.mobile.permission.MONEY_TRANSFER"`</code> with <code>`protectionLevel="signature"`</code> so only apps signed with the WellnessBank certificate can invoke this Activity. Add per-Intent caller verification in <code>`onNewIntent`</code> and <code>`onCreate`</code> using <code>`getCallingPackage()`</code> and comparing against an allowlist of trusted package signatures. Implement Intent extras schema validation before passing any amount or recipient field into the money-movement flow.	14d	No Control Found
T-3	Mobile IPC Input Validation (M4) — Intent Hijacking into Money Movement	Implement Intent extras schema validation in <code>`MoneyTransferActivity.onCreate`</code> and <code>`onNewIntent`</code> . Validate all expected extras (recipient account number format, transfer amount range, currency code enum) before any value is used in the money-movement flow. Reject the Intent and finish the Activity if any extra fails validation. Add <code>`android:permission`</code> with signature protection level (see E-2) as a layered control. The input validation must be server-side duplicated at the Backend API — the client-side validation is defense-in-depth but	14d	No Control Found

Priority	Finding	Recommendation	SLA	Status
		the backend MUST independently validate all transaction parameters before executing a transfer.		
T-4	Insufficient Mobile Binary Protections (M7)	Enable ProGuard/R8 code obfuscation in the release build configuration (<code>`minifyEnabled true`, `proguardFiles getDefaultProguardFile('proguard-android-optimize.txt')`</code>) to raise the cost of static reverse engineering. Add root-detection logic at app startup and before security-critical operations (transaction confirmation, payment authorization) using a multi-signal detection approach: check for <code>`su`</code> binary presence, test-keys build tag, write access to <code>`/system`</code> , and Magisk manager package. Integrate Play Integrity API (or RootBeer as a lighter alternative) to block operation on compromised devices. Add basic debugger-attach detection with <code>`android.os.Debug.isDebuggerConnected()`</code> at sensitive flow entry points.	14d	No Control Found
I-4	Insufficient Mobile Cryptography (M10)	Replace the custom 4-digit PIN PBKDF2 derivation with Android Keystore-backed AES-256-GCM key generation. The user's biometric or device PIN unlocks a hardware-backed key in the Keystore; no raw key material is ever exposed in the application process. If a PIN-based fallback is required, use PBKDF2-HMAC-SHA256 with at least 310,000 iterations (OWASP 2023 recommendation) and a cryptographically random 256-bit salt stored separately from the derived key material. Replace the credential cache encryption with EncryptedSharedPreferences backed by the Keystore master key.	14d	No Control Found
S-5	Long-Lived Session Token Replay	Harden the existing authentication session flow by enforcing short session token TTLs (15–30 minutes for financial app sessions per PCI DSS guidance), implementing token rotation on each authenticated request, adding device-binding to session tokens using the Android Keystore attestation key, and enforcing biometric re-authentication before money-movement operations. The current flow persists a long-lived token in <code>MODE_PRIVATE</code> SharedPreferences — replace with EncryptedSharedPreferences backed by a Keystore key that requires biometric confirmation to decrypt. Add server-side session validation that checks token age, device fingerprint, and rejects replayed tokens.	14d	Partial Control

Priority	Finding	Recommendation	SLA	Status
I-2	Inadequate Mobile Privacy Controls (M6)	Add `FLAG_SECURE` to all Activities that display financial data (account balances, transaction history, payment screens). This single-call addition prevents the Android system from including the Activity content in the recents thumbnail and blocks screen-recording apps from capturing financial data. Call `window.setFlags(WindowManager.LayoutParams.FLAG_SECURE, WindowManager.LayoutParams.FLAG_SECURE)` in the `onCreate` of all financial-data Activities, or apply it globally from the Application class via an `ActivityLifecycleCallbacks` listener.	14d	No Control Found
I-7	Insecure Mobile Data Storage (M9) — Credential Cache	Replace plain `SharedPreferences` (`MODE_PRIVATE`) with `EncryptedSharedPreferences` backed by an Android Keystore master key. The migration path: on first run after upgrade, read existing plaintext credentials, re-persist in encrypted form, and delete the plaintext file. All subsequent reads and writes use the encrypted API transparently. This addresses both I-7 (credential cache confidentiality) and contributes to S-1 and S-5 remediation.	14d	No Control Found
R-3	Deniable Financial Transactions	Implement transaction non-repudiation at the Backend API by generating an immutable, tamper-evident audit record for every money-movement event. Each record must include: authenticated user identity (sub from JWT), transaction timestamp (server-authoritative, not client-supplied), transaction parameters (amount, recipient, currency), originating session token fingerprint, and a server-side HMAC over the record using a key held in the backend's secret store. Store audit records in a separate append-only audit log table with no DELETE or UPDATE permissions granted to the API service account.	14d	No Control Found
MEDIUM 20 actions				
R-1	M8 Accountability-Loss: Mobile Audit Log Gaps	Replace all `Log.d("auth", ...)` statements that include username, token, or any PII with structured security event logging that routes to the Backend API audit trail. On the client, replace debug log calls with a no-op security event emitter in release builds (`BuildConfig.DEBUG` guard). Forward security-critical events (login success/failure, token refresh, permission denial, session expiry) to the backend audit log as part of the normal authenticated API flow.	30d	No Control Found

Priority	Finding	Recommendation	SLA	Status
I-8	Debug Log PII Leakage via Logcat	Remove all `Log.d` / `Log.v` calls that include credentials, tokens, usernames, or any PII from the production codebase. Apply a lint rule (`android.lint.disable` or a custom Lint check) that flags log calls with auth-adjacent strings as compile-time errors. In the release ProGuard configuration, add a rule to strip all `Log.d` and `Log.v` calls: <pre>-assumenosideeffects class android.util.Log { public static *** d(...); public static *** v(...); }</pre> .	30d	No Control Found
S-3	Credential Theft via SharedPreferences Extraction	Harden the existing <code>MODE_PRIVATE</code> <code>SharedPreferences</code> by migrating to <code>EncryptedSharedPreferences</code> (see I-7 recommendation). Additionally, set <code>android:allowBackup="false"</code> in the application manifest to prevent ADB backup extraction of the credential file on pre-Android-12 devices. Disable Google Drive auto-backup for the <code>SharedPreferences</code> directory by adding a <code>backup_rules.xml</code> that excludes <code>shared_prefs/credentials.xml</code> .	30d	Partial Control
D-2	Missing Rate Limiting on Backend Transaction API	Implement rate limiting on the Backend API transaction endpoints using a sliding-window rate limiter keyed on authenticated user ID. Configure separate limits for: read operations (balance/history queries), write operations (transfer initiation), and authentication attempts (login/token refresh). Add exponential backoff responses (HTTP 429 with <code>Retry-After</code> header) on limit breach. Implement global IP-based rate limiting at the API gateway or load balancer layer as a defense-in-depth measure against unauthenticated flooding.	30d	No Control Found
R-4	Unlogged Debug Activity Invocations	Add structured security event logging to <code>WellnessBankDebugActivity</code> capturing every invocation: timestamp, calling package (via <code>getCallingPackage()</code>), action name, and extras summary. Forward these events to the Backend API audit log. This is a defense-in-depth measure; the primary remediation for E-1 (Activity removal) is the correct fix — logging should accompany, not replace, removal.	30d	No Control Found
T-1	Mobile Supply Chain Integrity (M2) — Analytics SDK	Implement SDK supply chain integrity controls for the <code>WellnessAnalyticsSDK</code> . Pin the SDK dependency to an exact version with a cryptographic checksum verification in the Gradle dependency configuration. Add the SDK's published checksum (SHA-256) to <code>gradle.lockfile</code> or use Gradle dependency verification (<code>verification-metadata.xml</code>) to	30d	No Control Found

Priority	Finding	Recommendation	SLA	Status
		reject SDK artifacts with unexpected checksums. Establish an SDK update review process: before updating to a new SDK version, review the changelog and diff against the previous pinned checksum. Scope the SDK's manifest permissions to the minimum required.		
T-2	Mobile Supply Chain Integrity (M2) — Payment SDK	Apply the same supply chain integrity controls as T-1 to the WellnessPaySDK. Given the payment data sensitivity, apply additional scrutiny: verify the SDK publisher's PCI DSS compliance documentation, confirm the SDK is listed on the Google Play SDK Index with verified provenance, and implement a runtime attestation check that the loaded SDK binary matches the pinned checksum. Do not auto-update the payment SDK — require explicit engineering review for each version bump.	30d	No Control Found
I-3	Insecure Mobile Data Storage (M9) — Local Database	Replace the plain SQLite database (`WellnessBankLocalDB`) with SQLCipher for Android (`net.zetetic:android-database-sqlcipher`) to provide AES-256 encryption of the entire database file at rest. Generate the encryption key using Android Keystore and unlock it via biometric authentication. Additionally, set `android:allowBackup="false"` or add a `backup_rules.xml` excluding the database file to prevent Google Drive backup of the unencrypted database. Implement a maximum retention policy (e.g., purge transactions older than 90 days from local cache).	30d	No Control Found
T-6	Credential Cache Tampering via SharedPreferences Overwrite	Migrate from plain `SharedPreferences` to `EncryptedSharedPreferences` (see I-7 and S-3). Once encrypted, root-level file modification requires both root access AND knowledge of the Keystore-protected encryption key, significantly raising the bar for T-6's tampering scenario. Add integrity checking: on each credential load, verify a stored HMAC over the token value using a separate Keystore key to detect tampering.	30d	No Control Found
S-2	Insecure Mobile Authentication/Authorization (M3)	Harden the existing authentication flow by adding biometric re-authentication before all money-movement operations (`MoneyTransferActivity`, payment flows). Implement per-session re-authentication for sensitive actions using `BiometricPrompt` with a `CryptoObject` backed by a Keystore key. Add token expiry enforcement on the client (check token age before each	30d	Partial Control

Priority	Finding	Recommendation	SLA	Status
		API call; force re-login if expired). Implement risk-based authentication triggers: flag transactions above a threshold amount or to new payees for additional challenge.		
T-5	Unencrypted Local Database Tampering	SQLCipher encryption (see I-3) addresses both the confidentiality and integrity concerns for the local database. Additionally, set `android:allowBackup="false"` to prevent extraction of the database via Google Drive backup. As a defense-in-depth measure, implement application-level row-level integrity checks (store a HMAC over critical transaction record fields) to detect tampering even if the SQLCipher layer is bypassed.	30d	No Control Found
E-3	Server-Side Authorization Gap for Debug-Initiated Operations	Implement server-side request integrity validation at the Backend API that rejects requests originating from debug-Activity-initiated flows. Since the backend cannot directly inspect Android Activity provenance, the control must be architectural: issue short-lived, one-time-use request tokens bound to specific operations (transfer initiation, payment authorization) that are generated only after normal authenticated UI flows — not after debug Activity bypass. Consider Play Integrity API attestation tokens as a prerequisite for sensitive operations: the mobile client includes a fresh attestation token with each transaction request; the backend verifies it before processing.	30d	No Control Found
I-9	Backend API Verbose Error Response Exposure	Replace verbose error responses with generic client-facing error messages. Configure the Backend API error handler to return standardized error payloads (e.g., `{ "error": "BAD_REQUEST", "request_id": "uuid" }`) with no stack traces, field names, ORM details, or service internals. Log the full diagnostic information server-side (keyed by `request_id`) for internal debugging. Apply this to all error categories: validation errors, database errors, unexpected exceptions, and 404s.	30d	No Control Found
R-2	Missing Backend Audit Trail on Transaction Operations	Implement a Backend API audit trail for all transaction operations. Every state-changing operation (account read with PII, transfer initiation, transfer execution, payment authorization, balance inquiry) must generate an append-only audit log entry containing: authenticated user sub, operation type, resource identifier, request parameters (sanitized), outcome (success/	30d	No Control Found

Priority	Finding	Recommendation	SLA	Status
		failure), server timestamp, and source IP. Store audit logs in a write-only sink separate from the operational database (dedicated audit table with INSERT-only API service account, or external SIEM/log aggregation).		
S-1	Improper Mobile Credential Usage (M1)		30d	Partial Control
I-1	Insecure Mobile Communication (M5) — Client-to-Backend	Implement certificate pinning on the WellnessBank Backend API connection. Add a `network_security_config.xml` with a ` <pin-set>` declaration for the backend's leaf certificate or intermediate CA public key hash. Define a backup pin for rotation purposes. Set `expiration` to align with the certificate renewal schedule. Update the `android:networkSecurityConfig` attribute in the manifest. Implement a pin rotation strategy: when backend certificates are rotated, publish updated Network Security Config via app update before the old pin expires.</pin-set>	30d	Partial Control
D-4	Credential Store Corruption Leading to Denial of Service	The EncryptedSharedPreferences migration (I-7) raises the bar for T-6 tampering. For D-4 specifically, add defensive read-with-fallback logic in the credential cache: if the SharedPreferences file is unreadable, corrupt, or fails HMAC verification, initiate a graceful re-authentication prompt rather than a crash loop. Store a backup token refresh mechanism so the user is not permanently locked out if the credential cache is corrupted. Limit the blast radius of corruption to a re-login prompt.	30d	No Control Found
D-1	Client-Side Resource Exhaustion via Unconstrained SDK I/O	Implement SDK I/O budgets in the WellnessBank Android Client to constrain the network and battery resource consumption of embedded SDKs. Use `WorkManager` with quota constraints to schedule non-urgent SDK operations (analytics flushing, telemetry batching) rather than allowing real-time unbounded network calls. Set SDK initialization timeouts. Monitor battery impact via `BatteryManager` metrics in the Application health reporting.	30d	No Control Found
I-6	Insecure Mobile Communication (M5) — Payment SDK Egress	Extend certificate pinning to the WellnessPaySDK payment authorization flow (WellnessPaySDK → Third-Party Payment Provider). Since the SDK controls its own network client, the pinning must be implemented within the SDK or via an OkHttp `CertificatePinner` wrapper injected into the SDK's HTTP client at initialization. If the SDK does not expose a	30d	Partial Control

Priority	Finding	Recommendation	SLA	Status
		way to inject a custom HTTP client, configure Android's `network_security_config.xml` with a ` <domain >`="" appropriate="" domain="" entry="" for="" includesubdomains="true" key="" payment="" pin.<="" provider's="" public="" td="" the="" with=""><td></td><td></td></domain>		
D-3	Local Database Saturation	Implement a maximum row count or total size limit for the `WellnessBankLocalDB` cache. Configure a periodic background job (via `WorkManager`) to enforce retention policy: delete transaction records older than 90 days, keeping only the most recent N transactions. Add an initial capacity check at app startup to prevent the app from attempting to operate on an oversaturated database.	30d	No Control Found
LOW 2 actions				
I-5	Insecure Mobile Communication (M5) — Analytics SDK Egress	Add a ` <domain>` entry for the Third-Party Analytics Provider in `network_security_config.xml` with certificate pinning. Given that analytics data has lower sensitivity than payment or credential data, a single leaf certificate pin (with one backup pin) is sufficient. Coordinate with the analytics provider on their certificate rotation schedule.</domain>	90d	Partial Control
S-4	Missing Token Binding on Backend Session	Implement device binding for Backend API session tokens. When a session token is issued at login, record the requesting device's identifier (derived from Play Integrity attestation or a stable device fingerprint) alongside the token. On each subsequent API call, verify the token's device binding against the request's device identifier. Tokens used from a different device (or without a valid attestation) are rejected. This prevents token replay from a stolen credential with no device presence.	90d	Partial Control